Assignment 1: Branch Predictor Implementation

Daniel Prol - UH ID: 2446940

1. Summary

I implemented the `pm_predictor` class in `src/my_predictor.h`: a hybrid branch direction predictor based on the Pentium M, with a bimodal predictor and a tagged global predictor. As the assignment allows, the loop predictor, the branch target prediction and the PIR are not implemented (the global history register is used instead of the PIR).

Over the 20 traces of the infrastructure the predictor has an average of **5.848 MPKI**. The sample gshare predictor of the infrastructure has 6.306 MPKI with about twice the storage.

The class also reproduces the worked example of the handout (Appendix B): the 22 final predictions and the global hit / no hit column are the same.

2. Design

2.1 Bimodal predictor

A table of 4096 2-bit saturating counters indexed with the low 12 bits of the branch address. The counters start at 1 (weakly not taken). The counter of the branch is updated with every conditional branch outcome, also when the global predictor gave the final prediction.

2.2 Global predictor

A set-associative cache with 2048 entries: 512 sets and 4 ways. Each entry has a valid bit, a 6-bit tag, a 2-bit counter and an LRU age.

- Hash: the low 15 bits of the branch address XOR the 15-bit global history register.
- Set: the upper 9 bits of the hash. Tag: the lower 6 bits. This is the same split as the handout example (row = upper bits, tag = lower bits).
- Hit: a valid entry of the set has the same tag.

2.3 Final prediction

If the global predictor has a hit, the prediction is the upper bit of the counter of that entry. If not, it is the upper bit of the bimodal counter. Unconditional branches are predicted taken and do not change any state.

2.4 Update

- Global hit: the counter of the entry is updated with the outcome and the entry becomes the most recently used of its set.
- Global miss and the branch was mispredicted: a new entry is allocated. The victim is the first invalid way, or the least recently used way if all are valid. The counter of the new entry starts in the weak state of the outcome (2 if taken, 1 if not taken).
- Global miss and the branch was predicted correctly: nothing is allocated. The bimodal predictor is enough for this branch with this history, so the entry is left for a branch that needs it.
- The bimodal counter is updated and the outcome is shifted into the history register.

The information of the lookup (bimodal index, set, tag, hit way) is saved in `pm_update` by `predict` and used by `update`.

2.5 Storage

Component	Bits
Bimodal: 4096 x 2	8,192
Global: 2048 x (2 counter + 6 tag + 1 valid + 2 LRU)	22,528
History register	15
Total	30,735

The sample gshare has 32,768 2-bit counters = 65,536 bits.

3. Verification with the handout example

`src/test_example.cc` builds a small `pm_predictor` with the parameters of the example of Appendix B (16 bimodal counters, 2-way global predictor with 4 sets and 2-bit tags, 4-bit history, address shifted right by 2) and the same initial counter values, and runs the 22 branches of the table.

The example allocates a global entry on every global miss and the new entry keeps the counter of the entry it replaces. `pm_predictor` has this policy as an option (`alloc_on_any_miss = true`). With it, the set, the tag, the hit and the final prediction of the 22 rows are the same as in the handout:

```
$ cd src; make test_example; ./test_example
 1  set 0 tag 1  no hit  predict NT  actual T
 2  set 1 tag 1  no hit  predict NT  actual T
 3  set 0 tag 2  no hit  predict T   actual NT
 ...
20  set 2 tag 0  hit     predict T   actual T
21  set 2 tag 0  hit     predict T   actual T
22  set 1 tag 3  no hit  predict T   actual T
0 mismatches
```

4. Results

MPKI = direction mispredictions of conditional branches per 1000 instructions (each trace has 100 million instructions). The columns are:

- **gshare**: the sample predictor of the infrastructure (32,768 counters, 15 bits of history).
- **bimodal**: only the 4096-entry bimodal table.
- **PM any miss**: `pm_predictor`, 4-way, with the allocation of the handout example (allocate on every global miss, keep the old counter).
- **PM 2-way**: `pm_predictor` with a 2-way global predictor (1024 sets, 5-bit tag), allocation on misprediction.
- **PM 4-way**: the submitted configuration (the default of `pm_predictor`).

Trace	gshare	bimodal	PM any miss	PM 2-way	PM 4-way
164.gzip	12.473	15.030	12.690	12.458	12.436
175.vpr	13.415	14.671	15.965	13.519	13.298
176.gcc	11.254	19.347	18.006	9.534	9.285
181.mcf	15.837	33.245	18.264	15.927	15.621
186.crafty	5.837	13.075	9.101	5.351	5.211
197.parser	10.008	15.417	10.929	8.656	8.538
201.compress	7.831	9.356	8.014	7.748	7.702

Trace	gshare	bimodal	PM any miss	PM 2-way	PM 4-way
202.jess	1.562	8.809	2.001	1.134	1.118
205.raytrace	2.756	3.806	3.606	2.421	2.307
209.db	3.909	6.480	4.818	3.665	3.615
213.javac	2.267	3.181	2.996	2.061	2.037
222.mpegaudio	2.188	3.154	2.927	2.057	2.028
227.mtrt	2.657	3.706	3.439	2.419	2.383
228.jack	3.033	7.691	4.234	2.385	2.353
252.eon	1.807	9.405	2.145	1.470	1.425
253.perlbmk	2.554	5.955	5.120	1.864	1.819
254.gap	3.926	9.675	4.304	3.530	3.507
255.vortex	1.222	1.916	1.636	0.836	0.828
256.bzip2	0.094	0.112	0.110	0.058	0.057
300.twolf	21.489	21.942	24.816	21.529	21.389
Average	**6.306**	**10.299**	**7.756**	**5.931**	**5.848**

The PM 4-way column is the output of the original driver (`./run.sh traces`, file `results.txt`); all the columns are produced by `./compare.sh traces` (file `comparison.txt`).

4.1 Discussion

- The global predictor is what makes the hybrid work: the bimodal table alone has 10.299 MPKI and the hybrid 5.848 (43% less).
- The submitted predictor is better than the sample gshare in all the 20 traces, with 7% fewer mispredictions on average and less than half of the storage. The difference is large in the traces with many branches (176.gcc: 11.254 to 9.285). I did not measure the cause; the probable one is that the tags avoid the aliasing between branches that gshare has, and the bimodal table predicts the branches that do not need history.
- The allocation policy is the most important decision. With the allocation of the handout example (every global miss) the average is 7.756, worse than gshare: every branch executed replaces an entry, so the entries of the branches that need history are evicted by branches that the bimodal table already predicts well. Allocating only after a misprediction gives 5.848.
- 4 ways are a little better than 2 ways (5.848 against 5.931) with the same 2048 entries.
- 300.twolf, 181.mcf, 175.vpr and 164.gzip stay above 12 MPKI with all the predictors, and the global predictor helps little there (300.twolf: 21.942 with the bimodal table alone, 21.389 with the hybrid).

5. What is not implemented

- Loop predictor, branch target prediction (`target_prediction` is always 0) and the PIR hash; the assignment says these can be skipped.
- The extra credit class `cpm_predictor` is not changed.

6. How to run

```
cd src
make
cd ..
./run traces          # original script, needs csh and dc
./run.sh traces       # the same with bash
```

No external code was used.